

Evidencias

Alumno Jose Carlos De La Cruz Vazquez

Libreta

A.1 Conectivos

$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$	
p	q	r	p	q	r
1	0	1	1	1	1
0	1	0	1	0	0
	0	1	0	1	0
	0	1	0	0	1

Act. 2 Las 3 Obligatorias

$\neg(p \wedge q)$	$(p \vee q) \rightarrow r$	$p \vee q$	$(p \vee q) \rightarrow r$		
p	q	$\neg(p \wedge q)$	p	q	r
1	1	0	1	1	1
1	0	1	1	1	0
0	1	1	1	0	1
0	0	1	1	0	0
			0	1	1
			0	1	0
			0	0	1

$(p \leftrightarrow q) \vee \neg r$

p	q	r	$p \leftrightarrow q$	$\neg$	$(p \leftrightarrow q) \vee \neg r$
1	1	1	1	0	1
1	1	0	1	1	1
1	0	1	0	0	0
1	0	0	0	1	1
0	1	1	0	0	0
0	1	0	0	1	1
0	0	1	1	0	1
0	0	0	1	1	1

Act. 3 Clasificación

p	$\neg p$	$p \vee \neg p$	p	$\neg p$	$p \vee \neg p$
1	0	1	1	0	1
0	1	1	0	1	1

- Contradicción - tautología

$((p \rightarrow q) \wedge (p \rightarrow r)) \rightarrow (p \rightarrow r)$ tautologías

p	q	r	$p \rightarrow q$	$q \rightarrow r$	$\wedge$	$p \rightarrow r$	$\vee$
1	1	1	1	1	1	1	1
1	1	0	1	0	0	0	1
1	0	1	0	1	0	1	1
1	0	0	0	1	0	0	1
0	1	1	1	1	1	1	1
0	1	0	1	0	0	1	1
0	0	1	1	1	1	1	1
0	0	0	1	1	1	1	1

$(p \rightarrow q) \rightarrow (q \rightarrow p)$

p	q	$p \rightarrow q$	$q \rightarrow p$	$\vee$
1	1	1	1	1
1	0	0	1	1
0	1	1	0	0
0	0	1	1	1

Act. 5 Argumentos

1) $p \wedge q, p \wedge q$

$p \wedge q \rightarrow p \wedge q$

1 1 1 1 1

Valido modus ponens

2) $p \rightarrow q, \neg p \rightarrow q$

$p \wedge \neg q \rightarrow p \wedge \neg q$

0 1 1 1 0

Invalido
Falacia

3) $p \rightarrow q, \neg q \rightarrow \neg p$

$p \wedge \neg q \rightarrow p \wedge \neg q$

1 1 1 0 0

Valido
Modus tollens

4) $p \vee q, \neg p \rightarrow q$

$p \wedge \neg q \rightarrow p \wedge \neg q$

1 1 1 0 1

Valido
Silogismo
disyuntivo

Pruebas

- PS C:\Users\Vivob\Documents\practica3_logica> python .\conectivos.py

Resulatdo

```
PS C:\Users\Vivob\Documents\practica3_logica> python .\conectivos.py
negacion
p | not p
1 | 0
0 | 1

conjuncion
p | q | resultado
1 | 1 | 1
1 | 0 | 0
0 | 1 | 0
0 | 0 | 0

disyuncion
p | q | resultado
1 | 1 | 1
1 | 0 | 1
0 | 1 | 1
0 | 0 | 0

condicional
p | q | resultado
1 | 1 | 1
1 | 0 | 0
0 | 1 | 1
0 | 0 | 1

bicondicional
p | q | resultado
1 | 1 | 1
1 | 0 | 0
0 | 1 | 0
0 | 0 | 1

disyuncion_exclusiva
p | q | resultado
1 | 1 | 0
1 | 0 | 1
0 | 1 | 1
0 | 0 | 0
```

- PS C:\Users\Vivob\Documents\practica3_logica> python .\tabla.py

```
PS C:\Users\Vivob\Documents\practica3_logica> python .\tabla.py
```

```
~(p ^ q)
p | q | negacion(conjuncion(p, q))
-----
1 | 1 | 0
1 | 0 | 1
0 | 1 | 1
0 | 0 | 1

(p v q) -> r
p | q | r | condicional(disyuncion(p, q), r)
-----
1 | 1 | 1 | 1
1 | 1 | 0 | 0
1 | 0 | 1 | 1
1 | 0 | 0 | 0
0 | 1 | 1 | 1
0 | 1 | 0 | 0
0 | 0 | 1 | 1
0 | 0 | 0 | 1

(p <-> q) v ~r
p | q | r | disyuncion(bicondicional(p, q), negacion(r))
-----
1 | 1 | 1 | 1
1 | 1 | 0 | 1
1 | 0 | 1 | 0
1 | 0 | 0 | 1
0 | 1 | 1 | 0
0 | 1 | 0 | 1
0 | 0 | 1 | 1
0 | 0 | 0 | 1
```

- PS C:\Users\Vivob\Documents\practica3_logica> python .\clasificar.py

```
PS C:\Users\Vivob\Documents\practica3_logica> python .\clasificar.py

p ^ ~p
p | conjuncion(p, negacion(p))
-----
1 | 0
0 | 0
-> contradiccion

p v ~p
p | disyuncion(p, negacion(p))
-----
1 | 1
0 | 1
-> tautologia

((p -> q) ^ (q -> r)) -> (p -> r)
p | q | r | condicional(conjuncion(condicional(p, q), condicional(q, r)), condicional(p, r))
-----
1 | 1 | 1 | 1
1 | 1 | 0 | 1
1 | 0 | 1 | 1
1 | 0 | 0 | 1
0 | 1 | 1 | 1
0 | 1 | 0 | 1
0 | 0 | 1 | 1
0 | 0 | 0 | 1
-> tautologia

(p -> q) -> (q -> p)
p | q | condicional(condicional(p, q), condicional(q, p))
-----
1 | 1 | 1
1 | 0 | 1
0 | 1 | 0
0 | 0 | 1
-> contingencia
```

- PS C:\Users\Vivob\Documents\practica3_logica> python .\equivalencias.py

```
PS C:\Users\Vivob\Documents\practica3_logica> python .\equivalencias.py
Ley | Equivalentes
-----
De Morgan (1) | True
De Morgan (2) | True
Implicacion material | True
Contrapositiva | True
Distributiva | True
Doble negacion | True
Caso negativo (reciproca) | False
```

- PS C:\Users\Vivob\Documents\practica3_logica> python .\argumentos.py

```
PS C:\Users\Vivob\Documents\practica3_logica> python .\argumentos.py
1. p -> q, p |- q (modus ponens): VALIDO

Contraejemplo: {'p': False, 'q': True}
2. p -> q, ~p |- ~q (negar el antecedente): INVALIDO

3. p -> q, ~q |- ~p (modus tollens): VALIDO

4. p v q, ~p |- q (silogismo disyuntivo): VALIDO
```

- PS C:\Users\Vivob\Documents\practica3_logica> python .\argumentos.py

```
PS C:\Users\Vivob\Documents\practica3_logica> python .\argumentos.py
1. p -> q, p |- q (modus ponens): VALIDO

Contraejemplo: {'p': False, 'q': True}
2. p -> q, ~p |- ~q (negar el antecedente): INVALIDO

3. p -> q, ~q |- ~p (modus tollens): VALIDO

4. p v q, ~p |- q (silogismo disyuntivo): VALIDO
```

- PS C:\Users\Vivob\Documents\practica3_logica> python .\caso_sistema.py

```
PS C:\Users\Vivob\Documents\practica3_logica> python .\caso_sistema.py
p | q | r | R1 | R2
-----
1 | 1 | 1 | 1 | 1
1 | 1 | 0 | 0 | 1
1 | 0 | 1 | 1 | 0
1 | 0 | 0 | 1 | 1
0 | 1 | 1 | 1 | 1 <-- escenario buscado
0 | 1 | 0 | 1 | 1
0 | 0 | 1 | 1 | 0
0 | 0 | 0 | 1 | 1
```

- PS C:\Users\Vivob\Documents\practica3_logica> python .\reto.py

```
PS C:\Users\Vivob\Documents\practica3_logica> python .\reto.py
Escribe la formula (~ ^ v -> *): ~(p ^ q) -> r
Variables detectadas: ['p', 'q', 'r']
Traduccion: condicional(negacion(conjuncion(p, q)), r)
p | q | r | condicional(negacion(conjuncion(p, q)), r)
-----
1 | 1 | 1 | 1
1 | 1 | 0 | 1
1 | 0 | 1 | 1
1 | 0 | 0 | 0
0 | 1 | 1 | 1
0 | 1 | 0 | 0
0 | 0 | 1 | 1
0 | 0 | 0 | 0
```

Conclusión

Con esta práctica me quedó claro que las tablas de verdad no son solo un ejercicio de matemáticas: son exactamente lo que hace la computadora cada vez que evalúa un `if` en un programa. Cuando escribo una condición como `usuario_activo and tiene_permiso`, estoy armando una fórmula proposicional sin pensarlo, y los `and`, `or` y `not` son los mismos conectivos que combinamos con `p`, `q` y `r`. Incluso las leyes de De Morgan resultaron útiles de verdad, porque explican por qué `not (a and b)` se puede reescribir como `not a or not b` para hacer una condición más legible.

Lo que más me costó trabajo fue el condicional, porque en la vida diaria "si `p` entonces `q`" suena a causa y efecto, y cuesta aceptar que sea verdadero cuando `p` es falso. Lo resolví dejando de pensarlo como promesa y pensándolo como regla que solo se rompe en un caso: `p` verdadero con `q` falso. También me costó entender cómo `eval` convierte texto en código ejecutable, y ahí entendí por qué puede ser un riesgo de seguridad si no se controla lo que recibe.

En general, ver que unas cuantas líneas de Python pueden verificar un argumento o demostrar una equivalencia me hizo ver la lógica menos como teoría y más como una herramienta que ya uso sin darme cuenta.